

BAB IV

ANALISIS ALGORITMA REKURSIF

Efisiensi Algoritma

Tujuan utama analisis algoritma adalah untuk memahami efisiensi suatu algoritma ketika dijalankan. Efisiensi ini diukur dari dua sisi utama:

1. **Kompleksitas Waktu (*Time Complexity*):** Berapa lama waktu yang dibutuhkan algoritma untuk menyelesaikan masalah seiring bertambahnya ukuran input (n).
2. **Kompleksitas Ruang (*Space Complexity*):** Berapa banyak memori yang dibutuhkan algoritma untuk menyimpan data selama proses eksekusi.

Pada bab ini, analisis difokuskan pada algoritma rekursif menggunakan Relasi Rekurens.

4.1. Relasi Rekurens

Relasi rekurens (*recurrence relation*) adalah persamaan atau ketidaksamaan yang mendefinisikan suatu fungsi (seperti waktu berjalan $T(n)$ atau jumlah operasi $M(n)$) atau urutan secara implisit sebagai fungsi dari nilainya pada titik lain (input yang lebih kecil). Relasi rekurens merupakan metode utama dalam analisis algoritma yang menggunakan teknik rekursif.

Relasi rekurens memerlukan 2 komponen utama untuk menentukan solusi, yaitu:

1. **Langkah rekuren**
Merupakan persamaan yang mendefinisikan $T(n)$ berdasarkan $T(n - 1)$ atau $T(n/b)$, dan seterusnya.
2. **Kondisi awal (*Base Case*)**
Merupakan nilai yang dibutuhkan agar urutan dimulai, atau **kondisi penghenti** bagi fungsi rekursif. Hal ini memastikan bahwa rekursi akan "berhenti" ketika ukuran subproblem cukup kecil. Tanpa *base case*, akan ada tak hingga urutan yang memenuhi relasi rekurens tersebut atau akan mengalami *infinite looping*.

Langkah-langkah menetapkan relasi rekurens untuk analisis efisiensi dari suatu persamaan adalah:

1. Menentukan ukuran input (n).

2. Identifikasi operasi dasar algoritma (contoh: perkalian, perbandingan, atau perpindahan).
3. Menentukan relasi rekurens dan *base case* untuk jumlah eksekusi operasi dasar tersebut.
4. Menyelesaikan relasi rekuren untuk mendapatkan *order of growth* (Θ atau O).

Relasi rekurens dapat diklasifikasikan berdasarkan bentuknya:

1. **Homogen linear dengan koefisien konstan**

Berbentuk $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k}$. Contoh: Fibonacci $f_n = f_{n-1} + f_{n-2}$.

2. **Nonhomogen linear dengan koefisien konstan**

Memiliki fungsi tambahan $F(n)$ di sisi kanan yang tidak nol, seperti $H_n = 2H_{n-1} + 1$.

3. **Nonlinear**

Jika koefisiennya bukan konstanta (bergantung pada n), seperti $B_n = nB_{n-1}$.

Beberapa metode utama untuk menyelesaikan relasi rekurens adalah:

- Metode substitusi mundur
- Metode persamaan karakteristik (Order 2)

4.2. Penyelesaian Relasi Rekurens dengan Substitusi

Metode substitusi mundur (*Backward Substitution*) adalah cara iteratif untuk menyelesaikan relasi rekurens. Intinya adalah mencari solusi eksplisit (formula tertutup) dengan menggantikan ekspresi rekursif berulang kali hingga pola muncul.

Langkah-langkah penyelesaian:

1. **Mulai dari Persamaan Rekurens:** Tulis $T(n)$ awal.
2. **Substitusi Mundur:** Ganti $T(n-1)$ dengan definisinya, lalu $T(n-2)$, dst.
3. **Identifikasi Pola:** Rumuskan ekspresi umum $T(n)$ setelah i kali substitusi.
4. **Terapkan Base Case:** Cari nilai i agar mencapai kondisi basis, lalu substitusikan ke pola umum.

Contoh: Menghitung Faktorial

Relasi rekurens untuk jumlah perkalian $M(n)$ pada faktorial:

$$M(n) = M(n-1) + 1 \quad \text{untuk } n > 0, \quad M(0) = 0$$

Proses Substitusi:

1. $M(n) = M(n-1) + 1$
2. Substitusi 1: $M(n) = [M(n-2) + 1] + 1 = M(n-2) + 2$
3. Substitusi 2: $M(n) = [M(n-3) + 1] + 2 = M(n-3) + 3$

Pola Umum: $M(n) = M(n-i) + i$.

Solusi Akhir: Saat $i = n$ (mencapai $M(0)$), maka $M(n) = 0 + n \Rightarrow M(n) = n$.

Keterbatasan Metode Substitusi:

- Sulit jika pola yang muncul tidak teratur/rumit.
- Bermasalah dengan fungsi pembulatan (floor/ceiling).
- Hasil akhir idealnya harus dibuktikan dengan Induksi Matematika.

4.3. Relasi Rekurens Order 2

Relasi rekurens order 2 adalah relasi di mana suku ke- n bergantung pada dua suku sebelumnya. Bentuk umumnya:

$$a_n = c_1 a_{n-1} + c_2 a_{n-2}$$

Syarat: $c_2 \neq 0$. Dibutuhkan dua *base case* (a_0 dan a_1).

4.3.1 Relasi Rekurens Homogen Order 2

Penyelesaian dilakukan dengan mencari akar-akar dari **Persamaan Karakteristik**:

$$r^2 - c_1 r - c_2 = 0$$

Solusi Berdasarkan Akar Karakteristik (r):

1. **Kasus 1: Dua Akar Berbeda** ($r_1 \neq r_2$)

Solusi umum:

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n$$

Konstanta α_1 dan α_2 dicari dengan mensubstitusi *base case*.

2. **Kasus 2: Akar Kembar** ($r_1 = r_2 = r_0$)

Solusi umum:

$$a_n = \alpha_1 r_0^n + \alpha_2 n r_0^n$$

4.3.2 Relasi Rekurens Nonhomogen

Jika persamaan memiliki konstanta tambahan (tidak nol) di sisi kanan, misalnya:

$$A(n) = A(n-1) + A(n-2) + 1$$

Penyelesaiannya seringkali melibatkan transformasi agar menjadi homogen. Dalam kasus di atas, dengan memisalkan $B(n) = A(n) + 1$, persamaan dapat diubah menjadi bentuk Fibonacci biasa.

4.4. Studi Kasus Analisis Algoritma

Berikut adalah contoh penerapan analisis rekurens pada masalah algoritma klasik.

1. Tower of Hanoi

Masalah: Memindahkan n cakram dari tiang A ke tiang C dengan bantuan tiang B, tanpa meletakkan cakram besar di atas cakram kecil.

Tujuan Analisis: Mencari $M(n)$, yaitu **total jumlah langkah minimal** yang dibutuhkan.

Strategi Rekursif:

- Pindahkan $n - 1$ cakram dari A ke B (Biaya: $M(n - 1)$).
- Pindahkan 1 cakram terbesar dari A ke C (Biaya: 1 langkah).
- Pindahkan $n - 1$ cakram dari B ke C (Biaya: $M(n - 1)$).

Model Matematika:

$$M(n) = M(n-1) + 1 + M(n-1) \Rightarrow M(n) = 2M(n-1) + 1$$

Base Case: $M(1) = 1$.

Solusi Akhir: $M(n) = 2^n - 1$.

Kesimpulan: Kompleksitas algoritma ini adalah eksponensial. Penambahan 1 cakram akan melipatgandakan jumlah langkah.

2. Algoritma Fibonacci Naïve

Masalah: Menghitung bilangan Fibonacci ke- n menggunakan definisi $F(n) = F(n-1) + F(n-2)$.

Tujuan Analisis: Mencari $A(n)$, yaitu **jumlah operasi penjumlahan** yang dilakukan komputer.

Analisis Cost: Untuk menghitung $F(n)$, komputer melakukan rekursi ke kiri dan kanan, lalu menjumlahkan hasilnya (+1 operasi).

$$A(n) = A(n-1) + A(n-2) + 1$$

Base Case: $A(0) = 0, A(1) = 0$.

Solusi Akhir: $A(n) = F(n + 1) - 1$.

Kesimpulan: Biaya komputasi tumbuh seiring dengan besarnya nilai Fibonacci itu sendiri (Eksponensial). Algoritma ini sangat tidak efisien karena menghitung ulang sub-problem yang sama berkali-kali.